Eagle Delivery

COSC 461 | October 26 2026

*Part A: A Search and BFS Comparison**

For this project, the robot's position is represented by a row and column on the campus map. The robot starts at one building and tries to reach another. It can move up, down, left, or right, but it cannot move through walls or outside the map. Each move costs 1, so the goal is to find the shortest route to the destination. Other building entrances can be passed along the way.

I used Manhattan distance as the heuristic for A*, which adds the horizontal and vertical distance between the robot and its destination.

$h(s) = |\text{row} - \text{goal row}| + |\text{column} - \text{goal column}|$

This heuristic never overestimates the shortest path because the robot can only move one space at a time. BFS explores routes in order of length, while A* uses the heuristic to focus on routes closer to the goal. Both can find the shortest path, but A* may explore fewer locations.

Map	Task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S -> W	18	18	67	22
Baseline	L -> D	22	22	79	62
Baseline	H -> P	16	16	41	16
Detour	S -> W	26	26	70	51
Detour	L -> D	22	22	65	60
Detour	H -> P	16	16	41	16

S = Student Center; L = Halle Library; H = Pray-Harrold; D = Downing Hall; P = Putnam Hall; W = Walton Hall.

Looking at the results, both BFS and A* found the same shortest paths for all six tests. However, A* explored fewer locations each time. For example, on the baseline S -> W route, BFS explored 67 locations while A* only explored 22. When the detour was added, the S -> W route increased from 18 to 26 steps, while the other two routes stayed the same.

Part B: Q-Learning and Results

For Part B, I used Q-learning to help the robot learn the best route through training. I used epsilon-greedy to balance exploration and choosing the best-known action. With epsilon set to 0.20, the robot takes a random action 20% of the time and chooses an action with the highest Q-value the rest of the time.

The robot receives −1 for each action and an additional +40 when it reaches its destination. The Q-values are updated after each action based on the reward received and the estimated value of the next state.

Training and evaluation setting	**Value**
Training episodes	2,000 per seed
Seeds	0, 1, 2
Evaluation	100 episodes per seed, ε = 0 (no exploration)
Max steps	200 actions
Learning parameters	α = 0.3, γ = 0.95, constant training ε = 0.20
Slip probability	0

I trained the robot on three routes using emu_campus.txt, testing each route with three different seeds. The table below shows the average evaluation results.

Task	**Success mean ± SD**	**Return mean ± SD**	**Steps per success**
S → W	100% ± 0 pp	22.0 ± 0.0	18
L → D	100% ± 0 pp	18.0 ± 0.0	22
H → P	100% ± 0 pp	24.0 ± 0.0	16

After training, the robot successfully completed all three routes with a 100% success rate. The routes also matched the shortest paths found by BFS and A*. H -> P had the highest average reward because it required the fewest steps. All three seeds produced the same final results.

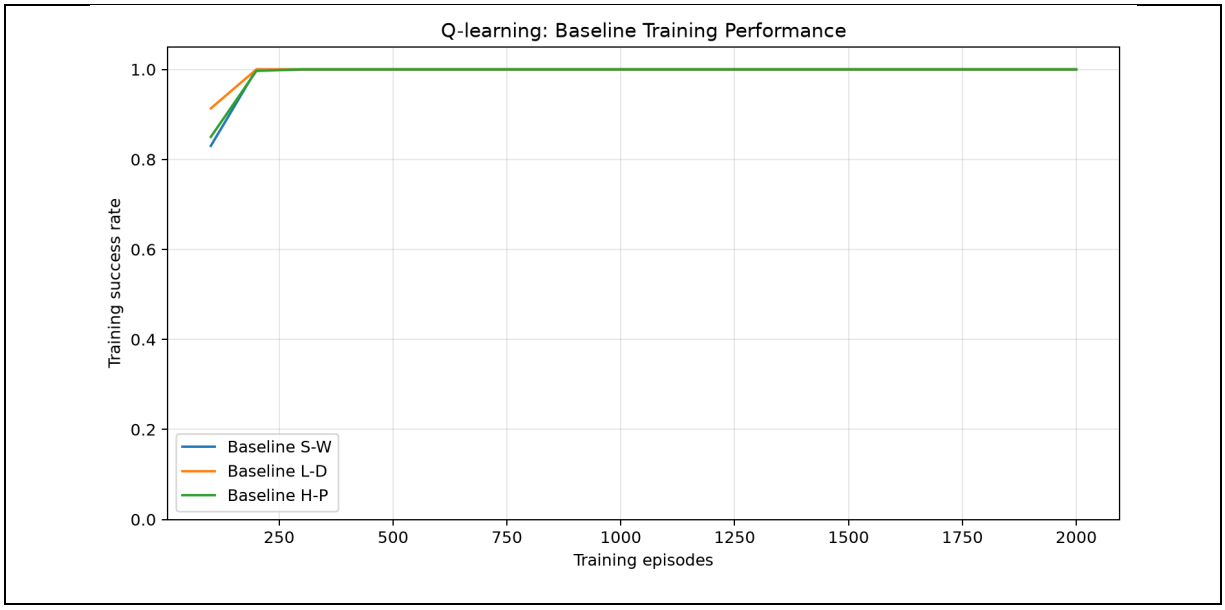


Figure 1. Training success rate over 2,000 episodes for the three delivery routes.

Part C: Custom Map and Exploration Experiment

For Part C, I wanted to see how different exploration rates would affect the robot's learning. I tested three epsilon values: 0.05, 0.20, and 0.50. I expected lower epsilon values to lead to better rewards because the robot would take fewer random actions.

I created a custom 9 x 13 campus map with all six building entrances, three dead ends, and a cycle. The map also includes walls and different possible routes to reach the destination. For the S -> W route, both BFS and A* found the shortest path of 10 steps, but BFS explored 35 locations while A* explored only 10.

I kept the same training settings from Part B and only changed epsilon. Each value was tested using three different seeds so I could compare how exploration affected the results.

Training ε	**Success (first 100 eps)**	**Avg. return (first 100)**	**Avg. return (episodes 401-500)**	**Final avg. steps**
0.05	96.00%	6.78	29.43	10
0.20	92.67%	2.09	27.22	10
0.50	93.00%	−3.63	18.84	10

Looking at the results, epsilon 0.05 had the highest average reward during training, while epsilon 0.50 had the lowest. This makes sense because a higher epsilon causes the robot to take more random actions. Even though the rewards were different, all three settings eventually reached a 100% success rate. After training, each setting found the same shortest route of 10 steps, with an average return of 30. This shows that exploration affected training performance but not the final route in my experiment.

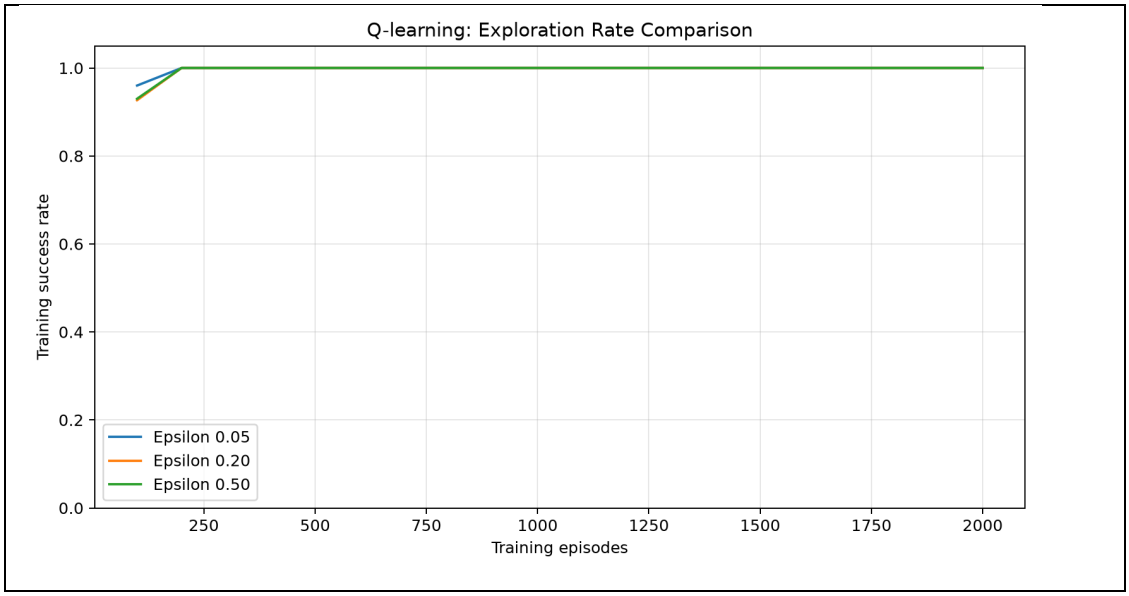


Figure 2. Training success rate for epsilon values 0.05, 0.20, and 0.50.